EXAM STUDY GUIDE / FALL 2026

운영체제 시험 대비 노트

2026년 가을학기 4190.307 | 강의·손필기 + xv6 과제 PA1–PA3

읽는 순서: 먼저 1부의 개념·계산을 복습하고, 2부에서 본인이 제출한 코드의 호출 경로와 설계 이유를 말로 설명해 본다. 각 문단의 '근거'는 원자료를 다시 찾기 위한 슬라이드 쪽수 또는 Git branch의 파일이다. 예상 문항은 출제 예측이지 교수자의 확정 범위가 아니다.

01	강의자료와 손필기 정리
02	과제 PA1–PA3 구현과 예상 문제

자료 기준 · pa1 / pa2 / pa3 로컬 branch + 2026년 가을학기 PDF 강의 슬라이드와 손필기

원자료 페이지와 코드 경로는 각 항목 아래에 표시했다. PDF 목차 창에서도 장별로 이동할 수 있다.

1부. 강의자료와 손필기 정리

1. 운영체제의 역할: 추상화, 자원 관리, 이벤트 처리

운영체제는 하드웨어를 응용 프로그램이 쓰기 편한 형태로 바꾸는 소프트웨어다. 여기서 커널은 권한 있는 명령과 장치·메모리·CPU를 직접 관리하는 핵심 부분이고, 컴파일러·셸·라이브러리·미들웨어 전체를 커널과 동일시하면 안 된다. 손필기에도 커널 경계와 라이브러리/프레임워크 층이 구분돼 있다.

- 응용 관점: CPU를 프로세스/스레드로, 물리 메모리를 사적 주소 공간으로, 저장 장치를 파일·디렉터리로 추상화한다. 같은 가상주소라도 서로 다른 프로세스에서는 다른 물리주소를 가리킬 수 있다.
- 시스템 관점: 한정된 CPU, 메모리, I/O, 큐, 에너지를 공유·보호하면서 공정성과 효율을 조절한다. '누구에게 줄 것인가'는 정책, '어떻게 전환할 것인가'는 기제다.
- 구현 관점: 커널은 시스템 콜과 인터럽트 같은 이벤트에 반응하는 동시성 소프트웨어다. 가상화, 동시성, 영속성은 과목의 세 축이다. 이번 자료와 PA1–PA3은 특히 CPU/메모리 가상화와 trap·스케줄링에 연결된다.

역사 흐름은 'I/O 동안 CPU가 놀지 않게 하자'에서 다중 프로그래밍이, '사용자에게 빨리 응답하자'에서 시분할이 나온 것으로 기억한다. 배치 시스템은 한 작업씩, 다중 프로그래밍은 여러 작업을 메모리에 두고 CPU 활용률을 높이며, 시분할은 선점·가상 메모리·대화형 셸을 이용해 응답성을 높인다. Unix의 `fork`/`exec`/`wait`와 파이프가 이후 과제 코드의 프로세스 모델이다.

> **시험 포인트** '운영체제 = 커널인가?'를 묻는다면 층을 나눠 답하라. 강의의 OS 정의는 넓은 시스템 소프트웨어 관점도 쓰지만, 보호 경계를 넘어 실행되는 코드는 커널이다.

근거 `0-overview.pdf` 15–17쪽; `1-intro.pdf` 3, 5–8, 14, 17–20쪽(손필기 포함).

2. 하드웨어 지원: I/O, 특권, trap, 타이머, 원자성

CPU와 장치는 병행 실행된다. PIO에서는 CPU가 장치와 메모리 사이 데이터 이동에 직접 관여한다. DMA에서는 장치 컨트롤러가 메모리와 블록을 옮기고 완료 시 인터럽트로 알린다. polling은 완료 여부를 계속 확인하므로 CPU를 낭비할 수 있다. 인터럽트는 장치·타이머에서 비동기적으로, 예외는 실행 중인 명령에서 동기적으로 발생한다. `ecall`은 의도적 예외(trap), page fault는 복구될 수 있는 fault다.

RISC-V 권한 수준은 M > S > U다. U-mode 응용은 CSR·PMP 같은 특권 자원에 직접 접근하지 못하고 `ecall`로 S-mode 서비스를 요청한다. S-mode도 M-mode 전용 PMP CSR을 직접 읽을 수 없으므로 PA2에서는 S→M 경계를 한 번 더 넘는다. CPU는 trap 원인(`scause`/`mcause`)과 복귀 PC(`sepc`/`mepc`)를 기록하고 trap 벡터(`stvec`/`mtvec`)로 간다. 시스템 콜의 `ecall`로 돌아올 때 PC를 4 증가시키지 않으면 같은 명령을 다시 실행한다. 반면 복구 가능한 page fault에서는 원인 명령을 재시도해야 한다.

협조적 스케줄링에서는 프로세스가 시스템 콜·`yield`로 자발적으로 CPU를 내준다. 무한 루프에서 커널이 제어권을 회수하려면 특권 타이머 인터럽트에 기반한 선점이 필요하다. 커널의 공유 변수 갱신은 LOAD–ADD–STORE처럼 여러 명령으로 쪼개지므로 다른 hart/인터럽트와 겹치면 lost update가 생긴다. 멀티코어에서 단순히 현재 코어의 인터럽트를 끄는 것만으로는 다른 코어를 막지 못한다. 원자적 RMW, spinlock 등의 동기화가 필요하다.

> **시험 포인트** `ecall`·타이머 인터럽트·page fault의 발생 시점, PC 복귀 위치, 처리 후 프로세스 상태를 각각 설명할 수 있어야 한다.

근거 `2-arch.pdf` 3–6, 8–12, 14–17, 19–25쪽(손필기 포함); PA2 명세 Background.

3. 프로세스와 문맥 교환

프로그램은 저장된 정적 코드이고 프로세스는 실행 중인 인스턴스다. 프로세스에는 PC·범용 레지스터 같은 CPU 문맥, 주소 공간, 열린 파일, PID, 상태 등이 포함된다. PCB(`struct proc`)가 이를 기록한다. 한 CPU에서 여러 프로세스가 번갈아 실행되므로 각각은 자기 CPU를 가진 것처럼 보인다.

상태 흐름은 `USED/created → RUNNABLE(ready) → RUNNING → RUNNABLE`(시간 조각 만료), `RUNNING → SLEEPING`(이벤트 대기), `SLEEPING → RUNNABLE`(깨움), `RUNNING → ZOMBIE`(종료)다. 'ready'와 'blocked'를 모두 실행 안 하는 상태라고 뭉뚱그리면 틀린다. ready는 CPU만 기다리고, blocked는 이벤트/다음 주기를 기다린다. PA3의 `sched_yield()`는 RT 프로세스를 후자로 바꾼다.

`fork()`는 새 PCB·주소 공간·파일 참조를 마련해 자식을 만들고 부모와 자식에게 다른 반환값을 준다. `exec()`는 기존 PID를 유지한 채 프로그램 이미지와 문맥을 새 프로그램으로 교체한다. 셸은 보통 fork → 자식 exec → 부모 wait 순서다. 종료했지만 수거되지 않은 자식은 zombie다. PA3에서는 자식의 RT 필드를 `allocproc()`에서 0으로 초기화해 RT 속성을 상속하지 않도록 했다.

문맥 교환에서는 trap으로 사용자 레지스터를 trapframe에 저장하고, 현재 프로세스의 커널 문맥을 PCB에 저장한 다음 스케줄러가 다음 프로세스의 문맥을 복원한다. `swtch`는 커널 문맥 전환이며 U↔S trap 자체와 같은 단계가 아니다. 페이지 테이블/TLB 전환 비용도 있을 수 있다. 정책은 '다음 대상 선택', 기제는 `trap`, 상태 변경, `swtch`, 복귀다.

> **시험 포인트** "타이머 인터럽트 후 A에서 B로 바뀌는 경로"를 uservec → usertrap → yield/sched → scheduler → swtch → B 복귀로 말하고 trapframe과 `context`의 역할을 구분하라.

근거 `3-process.pdf` 3–4, 8–22쪽(손필기 포함); `pa3:kernel/proc.c`, `pa3:kernel/trap.c`.

4. CPU 스케줄링: 평가 기준과 대표 정책

반환 시간(turnaround) = 완료 시각 − 도착 시각. 응답 시간(response) = 첫 실행 시각 − 도착 시각. 대기 시간은 ready queue에서 기다린 시간이며 I/O 대기와 구분한다. 같은 도착 시각의 세 작업 A=100, B=10, C=10이면 FIFO(A→B→C)의 평균 반환 시간은 (100+110+120)/3=110, SJF(B→C→A)는 (10+20+120)/3=50이다. 긴 작업 앞에 짧은 작업이 줄 선 convoy effect를 수치로 설명할 수 있어야 한다.

- FIFO/FCFS: 먼저 온 순서, 비선점, 간단하지만 convoy effect가 크다.
- SJF: 실행 시간이 알려지고 동시에 도착한다면 평균 반환 시간을 최소화한다. 비선점이고 긴 작업이 굶을 수 있다.
- STCF/SRTF: 남은 실행 시간이 가장 짧은 작업을 선택하는 선점형 SJF. 새 작업 도착 시 현재 작업의 남은 시간과 비교한다.
- RR: 각 작업에 time quantum을 주는 선점형 순환. 응답성이 좋아지지만 너무 작은 quantum은 문맥 교환 비용, 너무 큰 quantum은 FIFO에 가까운 지연을 만든다. 강의 예의 동일한 세 작업 A=B=C=30, q=10이면 RR 평균 응답 시간 (0+10+20)/3=10, 비선점 SJF는 30이다.
- 정적 우선순위: 높은 우선순위가 먼저, 동순위 내부는 FIFO/RR 가능. 낮은 우선순위의 starvation을 피하려면 aging이 필요하다.

MLFQ는 여러 우선순위 큐를 두고 실제 CPU 사용 행태로 우선순위를 갱신한다. 새 작업을 최상위에 넣고 시간 할당량을 다 쓰면 내린다. 자주 CPU를 내주는 대화형 작업을 우대하지만, 의도적으로 quantum 직전에 yield하면 우선순위를 유지하는 악용이 가능하다. 그래서 한 번의 연속 실행 시간이 아니라 해당 레벨의 누적 allotment를 회계하고, 주기적 priority boost로 장기 기아를 줄인다. I/O가 섞인 작업은 각 CPU burst를 나눠 보면 응답성을 이해하기 쉽다.

Linux CFS는 nice 값에서 weight를 얻고, 실제 실행 시간 증가량을 `weight(0)/weight(task)`로 정규화해 `vruntime`을 누적한다. 더 높은 weight는 같은 실제 실행 시간에도 vruntime이 느리게 늘어난다. 실행 가능 작업을 vruntime 기준

red-black tree에 보관하고 가장 작은 값을 선택한다. 예: weight 1024와 820의 이론적 점유율은 55.5%와 44.5%다. 이 일반 강의의 CFS와 PA3의 EDF는 다른 정책이다.

> **시험 포인트** 정책마다 최적화하는 기준, 선점 여부, starvation 가능성을 말로 비교하고, 숫자 문제에서는 도착·첫 실행·완료 시각을 먼저 적는다.

근거 `4-sched.pdf` 2–29쪽(손필기 포함), 특히 5–11, 15–20, 25–29쪽.

5. 실시간 EDF와 강의 스케줄링의 연결

주기 작업 `P_i=(C_i,T_i)`에서 C는 최악 실행 시간, T는 주기이며 과제 명세에서는 상대 deadline도 T다. ready인 RT 작업 중 절대 deadline이 가장 이른 작업이 우선이고, RT가 하나라도 ready이면 일반 RR 작업은 실행되지 않는다. 이상적 단일 CPU, 선점 가능, 독립 주기 작업이라는 명세 모델에서는 `U=Σ(C_i/T_i)≤1`이 EDF의 스케줄 가능 조건이다. 이는 과제 구현의 입장 허가 검사나 실행 시간 강제 장치가 아니다.

PA3 `task1`: (1,3)+(1,4)의 U=1/3+1/4=7/12 0.583. `task2`: (1,8)+(2,5)+(4,10)의 U=1/8+2/5+4/10=0.925. 두 집합은 이론적 조건을 만족한다. 도착/설정 시각이 t라면 첫 절대 deadline은 `t+T`다. 작업을 일찍 끝내도 다음 주기는 완료 시각이 아니라 이전 주기 경계에서 시작한다.

> **시험 포인트** EDF의 '가장 가까운 deadline'을 `C`가 짧은 작업, `T`가 짧은 작업, PID가 작은 작업과 혼동하지 말 것. PID는 deadline이 같은 경우의 2차 규칙이다.

근거 `4-sched.pdf` 23–25쪽; PA3 명세 Background와 Problem specification.

6. 가상 메모리: 왜 필요하고 어떤 설계가 다른가

가상 메모리는 프로세스마다 크고 연속적인 사적 주소 공간을 보여 주며 주소 변환, 보호, 유연한 배치, 필요할 때만 물리 메모리 할당을 가능하게 한다. 코드/데이터는 보통 `exec()` 때 배치되고 heap/stack은 동적으로 변한다. 소프트웨어 정적 relocation은 로드 시 주소를 고치지만 이후 재배치·보호가 어렵다. 동적 relocation은 MMU가 각 메모리 참조에서 변환한다.

손필기에서 강조한 '주소 공간의 과잉 약속(overcommit)'은 예약한 가상주소 전체에 물리 frame을 즉시 붙일 필요가 없다는 뜻이다. 접근 시점의 lazy allocation과 page fault 처리가 이를 가능하게 한다. 반대로 주소 공간을 예약했다는 사실만으로 실제 물리 메모리나 swap이 항상 충분하다는 보장은 없다.

고정 크기 partition은 `PA=base+VA`로 단순하지만 프로세스가 partition보다 작으면 내부 단편화가 발생한다. 가변 크기 partition은 base+limit로 경계를 검사하고 내부 낭비를 줄이지만 물리 메모리에 흩어진 빈 공간 때문에 외부 단편화가 생긴다. segmentation은 코드/데이터/heap/stack 같은 논리 단위를 `<segment,offset>`으로 나눠 각자 base·limit·보호를 적용한다. 공유·독립 성장에 유리하지만 각 segment의 연속 물리 공간이 필요해 외부 단편화가 남는다.

Paging은 고정 크기 페이지와 frame으로 공간을 나눈다. 프로세스의 페이지는 서로 떨어진 frame에 배치할 수 있어 외부 단편화가 없고 할당·반납이 쉽다. 마지막 페이지의 남는 부분은 내부 단편화이며 PTE 저장 공간과 변환 비용이 든다. 'segmentation은 논리적 가변 크기, paging은 동일 크기의 물리 배치 단위'라는 대조를 기억한다.

세그먼트를 프로세스끼리 공유할 때는 같은 물리 내용을 서로 다른 가상주소/세그먼트 번호에 놓을 수 있다. 그 내용 안에 저장된 포인터는 상대 프로세스에서 그대로 유효하지 않을 수 있으므로 같은 번호/주소에 놓거나 간접 주소 지정을 써야 한다. 이는 뒤의 `mmap` 공유 메모리에서 나타나는 포인터 문제와 같다.

근거 `5-vm.pdf` 2–20쪽; `5-vm_spring.pdf` 2–6, 15–20쪽(손필기 포함); `6-paging.pdf` 2–3, 12–13쪽.

7. Paging과 page fault 계산

페이지 크기가 `2^d` 바이트면 주소의 아래 d비트는 offset이다. `VPN = VA >> d`, `offset = VA & (2^d-1)`, `PA = (PFN << d) | offset`. 4KiB 페이지에서 offset은 12비트다. 32비트 VA라면 VPN은 20비트, 단일 선형 테이블은 `2^20`개의 PTE를 요구한다. PTE가 4바이트라면 프로세스당 `2^20×4=4MiB`다. 이 계산은 가상주소 전체에 PTE를 깔았을 때의 비용이다.

PTE에는 물리 frame 번호와 valid, 참조(R/A), dirty(M/D), 읽기·쓰기·실행, 사용자 권한 등이 있다. 일반 강의의 R/M 표기와 RISC-V의 A/D 명칭을 연결하라. invalid PTE 접근이 모두 같은 뜻은 아니다. 합법적인 페이지가 메모리에 없으면 major fault로 디스크 I/O가 필요할 수 있고, demand-zero/lazy allocation이면 minor fault로 새 frame만 준비할 수 있다. 애초 주소 공간 밖이거나 권한 위반이면 프로세스 오류다. fault 처리 후 PTE를 고쳐 같은 명령을 재시도한다.

과제 코드의 `usertrap()`에는 U-mode `ecall`(`scause=8`)과 page fault(`scause=13` load, `15` store)를 가르는 분기가 있다. `ecall`에서는 PC를 4 늘리지만, 복구 가능한 fault에서는 `vmfault()` 후 원래 명령으로 복귀한다. 이 대조는 PA2와 가상 메모리를 잇는 핵심 문제다.

근거 `6-paging.pdf` 4–13쪽(손필기 포함); `8-tlb.pdf` 12–15쪽; `pa3:kernel/trap.c`.

8. 다단계 페이지 테이블과 Sv39

선형 페이지 테이블은 드문드문 쓰는 큰 가상주소 공간에서 낭비가 크다. 다단계 테이블은 사용하지 않는 주소 범위의 하위 테이블을 만들지 않는다. 대신 TLB miss 때 여러 단계 PTE를 읽는 비용이 생긴다. IA-32 예에서 4KiB page, 4바이트 PTE면 한 page table에 1024개 엔트리(10비트)가 들어가므로 VA는 10비트 directory + 10비트 table + 12비트 offset으로 나뉜다.

xv6의 RISC-V Sv39는 39비트 VA를 `VPN[2](9) | VPN[1](9) | VPN[0](9) | offset(12)`로 나눈다. 4KiB 페이지에 8바이트 PTE 512개가 들어가므로 각 단계 인덱스가 9비트다. `satp`는 root page table의 물리 page 번호를 가리킨다. 세 번의 테이블 참조와 실제 데이터 참조가 필요한 경우 TLB가 매우 중요하다. 64비트 레지스터를 쓴다고 모든 64비트 가상주소를 Sv39가 변환하는 것은 아니다.

역페이지 테이블은 물리 frame마다 `<PID,VPN>`을 보관해 공간을 물리 메모리 크기에 비례시킨다. 대신 번역 시 탐색/해시가 필요하다. 다단계는 가상주소의 빈 부분을 줄이는 방식, 역페이지 테이블은 매핑 방향 자체를 바꾸는 방식이다.

근거 `7-pt.pdf` 2, 7–16쪽(손필기 포함); `6-paging.pdf` 13쪽.

9. TLB: miss는 곧 page fault가 아니다

TLB는 최근 VPN→PFN/PTE 변환의 하드웨어 캐시다. TLB hit이면 권한을 확인하고 바로 물리주소를 만든다. miss이면 페이지 테이블을 찾아본다. 거기에 유효한 매핑이 있으면 TLB를 채운 뒤 계속 실행하고, 없거나 권한이 맞지 않을 때 page fault가 발생한다. 하드웨어 관리 TLB에서는 MMU가 테이블을 walk하고, 소프트웨어 관리 TLB에서는 miss 자체로 OS가 개입한다. RISC-V Sv39은 하드웨어 page walk의 예다.

문맥 교환 시 다른 주소 공간의 같은 VPN을 잘못 쓰지 않도록 TLB를 비우거나 ASID로 엔트리를 구별한다. 페이지 테이블을 바꿔도 각 코어의 TLB에는 오래된 변환이 남을 수 있다. 다른 코어에 IPI를 보내 무효화시키는 것이 TLB shootdown이다. TLB reach(coverage)는 엔트리 수 × page 크기다. 예를 들어 64엔트리, 4KiB면 256KiB이고 2MiB hugepage를 쓰면 같은 엔트리 수로 훨씬 넓은 범위를 덮는다.

> **시험 포인트** `TLB miss / page fault / protection fault`를 순서대로 그려 설명하고, PTE만 고친 뒤 다른 코어가 왜 옛 물리주소를 쓸 수 있는지 말할 수 있어야 한다.

근거 `8-tlb.pdf` 2–15쪽(손필기 포함), 특히 7–10, 12–15쪽.

10. mmap, 공유 메모리, copy-on-write

`mmap`은 가상주소 범위에 파일·장치·공유 메모리·anonymous backing을 연결한다. 파일 mapping에서는 load/store가 파일 데이터에 접근하는 방식이 된다. 초기 PTE를 invalid로 두고 첫 접근에 페이지를 읽는 lazy 방식이 가능하다. `MAP_SHARED` 수정은 다른 공유자에게 보이며 파일에 반영될 수 있고, `MAP_PRIVATE`는 수정 시 개인 사본을 만들어 원본 파일을 유지한다. 파일 mapping과 anonymous mapping은 backing의 종류, shared/private는 변경 전파 방식이다.

공유 메모리는 서로 다른 프로세스의 PTE가 같은 물리 frame을 가리킨다. 각 프로세스의 가상주소는 달라도 된다. 이때 공유 영역 안의 절대 포인터는 다른 주소 공간에서 유효하지 않을 수 있다. COW `fork()`는 처음에 부모·자식이 동일한 읽기 전용 frame을 공유하고, 쓰기 protection fault 때 새 frame을 복사해 각자 쓰게 한다. 따라서 '공유'와 '사적 변경'이 동시에 가능하다.

`read` 기반 I/O에는 커널/사용자 공간 간 복사 등이 들어갈 수 있다. `mmap`은 포인터 접근으로 경로를 단순화하고 사본을 줄일 수 있지만 데이터 이동 시점의 직접 제어가 약하고 pipe/socket 같은 스트림 I/O에 그대로 적용되지 않는다.

근거 `9-mmap.pdf` 2–15쪽(손필기 포함), 특히 4–5, 7–14쪽.

11. Swapping과 page replacement

RAM을 디스크 페이지의 cache로 보아 필요한 페이지만 보관한다. free frame 수가 low watermark 아래면 page daemon이 회수하고 high watermark 위면 멈추는 방식이 제시된다. 희생 페이지가 dirty이면 저장하고, 깨끗하며 backing에 원본이 있으면 버릴 수 있다. page fault의 디스크 비용은 매우 크므로 작은 miss율 차이도 성능에 큰 영향을 준다.

- OPT(MIN): 앞으로 가장 오래 안 쓸 페이지를 제거한다. 미래를 알아야 하므로 실제 구현보다 하한 기준이다.
- FIFO: 가장 오래 메모리에 있던 페이지를 제거한다. 간단하지만 자주 쓰는 오래된 페이지도 쫓고, frame을 늘렸는데 fault가 늘어나는 Belady anomaly가 가능하다.
- LRU: 가장 오래 참조되지 않은 페이지를 제거한다. 참조 이력을 추적해야 하며 stack property가 있어 frame 수 증가로 fault가 늘지 않는다. 어떤 workload에서나 OPT에 가까운 것은 아니다.
- CLOCK: 원형 frame과 reference bit 사용. 손이 만난 페이지의 R=1이면 0으로 바꾸고 넘기고, R=0이면 교체한다. dirty bit까지 보면 깨끗한 후보를 우대할 수 있다.

강의의 FIFO 반례 참조열 `1 2 3 4 1 2 5 1 2 3 4 5`는 3 frames에서 9 faults, 4 frames에서 10 faults다. 숫자만 외우기보다 FIFO 큐를 직접 갱신할 수 있어야 한다. 전체 working set이 RAM에 안 들어와 거의 계속 page-in/out하면 thrashing이다. local replacement는 각 프로세스 몫 안에서, global replacement는 전체 frame에서 고르므로 프로세스끼리 서로 영향을 준다.

근거 `10-swap.pdf` 2–20쪽(손필기 포함), 특히 6–8, 9–13, 17–20쪽.

12. 강의 계산·서술 자가 점검

문제 A. 32비트 VA, 4KiB page, 4바이트 PTE의 선형 테이블 크기는? 답: offset 12비트, VPN 20비트, `2^20×4=4MiB`.

문제 B. Sv39 VA의 분해와 `satp`의 역할은? 답: 9+9+9+12비트; root 페이지 테이블의 PPN 제공.

문제 C. TLB miss 뒤에도 page fault가 없을 수 있는가? 답: 있다. PTE가 유효하면 page walk로 TLB를 채운다.

문제 D. `fork` 직후 COW page에 자식이 쓸 때의 순서는? 답: 읽기 전용 공유 page에 write fault → 새 frame 할당·복사 → 자식 PTE를 writable 사본으로 교체 → 명령 재시도.

문제 E. SJF, RR, EDF가 각각 비교하는 것은? 답: 전체/남은 CPU 시간, 순환 순서와 quantum, 절대 deadline. 서로 다른 목표의 정책이다.

2부. 과제 PA1–PA3 구현과 예상 문제

제출 브랜치 pa1(c236869), pa2(9ccbcf4), pa3(f42c080)의 실제 코드를 인용했다. 줄번호는 각 브랜치 기준이다. 코드의 ▌ 표시는 지면에서 긴 한 줄을 이어 쓴 표시이며 소스 문자는 아니다. 과제 지시문의 기준은 SNU CSL 공식 PA1·PA2·PA3 저장소다.

읽는 법: 각 과제의 요구사항 → 실행 경로 → 제출 코드 → 시험에서 설명할 근거 순서로 본다. 코드는 파일 전체가 아니라 관련 구간을 정확히 발췌했다.

2.1 PA1 — 부팅 메시지와 커널 초기화

무엇을 했나

커널 부팅 때 출력되는 기본 안내 뒤에 학기 문자열과 학번·이름을 나타내는 문자를 출력했다. 핵심은 사용자 프로그램이 시작되기 전, hart 0의 kernel/main.c:main() 안에서 printk()가 실행된다는 점이다.

pa1:kernel/main.c — hart 0의 부팅 메시지

```
13    if (cpuid() == 0) {
14      consoleinit();
15      printkinit();
16      printk("\n");
17      printk("xv6 kernel is booting\n");
18      printk("\n");
19
20      printk("SNUOS2026\n");
21      printk("\n");
```

그 뒤의 여러 printk()는 학번 ASCII 그림과 이름 그림을 그린다. 시험에서 출력 위치를 묻는다면 consoleinit()과 printkinit() 뒤, kinit()/kvminit()/userinit() 앞이라고 답한다.

pa1:kernel/main.c — 그림 출력 뒤 초기화 순서

```
26    printk("▌▌▌▌▌▌▌ ▌▌▌▌ ▌▌▌▌▌▌▌  ▌▌    ▌▌▌▌ ▌▌▌▌ ▌▌▌▌ ▌▌▌▌ ▌▌▌▌\n");
27    printk("\n");
28    printk("▌▌▌▌▌▌▌ ▌▌▌▌ ▌▌▌▌▌▌▌  ▌▌    ▌▌▌▌ ▌▌▌▌ ▌▌▌▌ ▌▌▌▌ ▌▌▌▌\n");
29    printk("▌▌▌▌▌▌▌ ▌▌▌▌ ▌▌▌▌▌▌▌  ▌▌    ▌▌▌▌ ▌▌▌▌ ▌▌▌▌ ▌▌▌▌ ▌▌▌▌\n");
30    printk("▌▌▌▌▌▌▌ ▌▌▌▌ ▌▌▌▌▌▌▌  ▌▌    ▌▌▌▌ ▌▌▌▌ ▌▌▌▌ ▌▌▌▌ ▌▌▌▌\n");
31    printk("▌▌▌▌▌▌▌ ▌▌▌▌ ▌▌▌▌▌▌▌  ▌▌    ▌▌▌▌ ▌▌▌▌ ▌▌▌▌ ▌▌▌▌ ▌▌▌▌\n");
32    printk("▌▌▌▌▌▌▌ ▌▌▌▌ ▌▌▌▌▌▌▌  ▌▌    ▌▌▌▌ ▌▌▌▌ ▌▌▌▌ ▌▌▌▌ ▌▌▌▌\n");
33    printk("▌▌▌▌▌▌▌ ▌▌▌▌ ▌▌▌▌▌▌▌  ▌▌    ▌▌▌▌ ▌▌▌▌ ▌▌▌▌ ▌▌▌▌ ▌▌▌▌\n");
34
35
36
37    kinit();         // physical page allocator
38    kvminit();       // create kernel page table
39    kvminithart();   // turn on paging
40    procinit();      // process table
```

실행 흐름·예상 문제

- **부팅 순서:** entry.S → start() → main() → consoleinit()/printkinit() → printk() → 메모리·trap·디스크 초기화 → userinit() → scheduler().
- **왜 hart 0만?** main()의 cpuid()==0 분기에서 전역 커널 초기화를 한 번 수행한다. 나머지 hart는 started 플래그를 기다린 뒤 자기 hart의 MMU·trap·PLIC 설정을 한다.
- **왜 printf가 아니라 printk?** 커널 부팅 중 커널 로그용 출력 경로를 이용한다. 이 시점에는 사용자 프로세스가 없고 user printf()/write() 경로를 쓸 수 없다.

Q. 문자열이 userinit() 뒤에 있으면 어떤 차이가 생기는가?

A. 출력 시점이 늦어져 부팅 초기 상태를 보여 주지 못한다. 사용자 프로세스가 만들어진 뒤의 메시지가 되므로 초기화 단계의 실패 위치를 분리해 보기 어렵다.

2.2 PA2 — 시스템 콜과 PMP 레지스터 조회

목표와 호출 경로

세 사용자 API는 nenter()=23, getpmpaddr(n)=24, getpmpcfg(n)=25이다. nenter는 콘솔 입력에서 받아들인 Enter의 누적 수를 반환한다. 나머지 둘은 S-mode 커널이 직접 읽을 수 없는 M-mode PMP CSR 값을 가져온다.

U-mode 사용자 호출 → ecall → S-mode usertrap/syscall → sysproc.c → (PMP일 때) S-mode ecall → M-mode machinevec → CSR 읽기 → mret → S-mode → sret → U-mode. 두 번의 ecall은 출발 모드와 trap 대상이 다르다.

pa2:kernel/syscall.h — 번호

```
25  #define SYS_nenter      23
26  #define SYS_getpmpaddr  24
27  #define SYS_getpmpcfg   25
```

pa2:kernel/syscall.c — 디스패치 표

```
138   [SYS_nenter]  = sys_nenter,
139   [SYS_getpmpaddr] = sys_getpmpaddr,
140   [SYS_getpmpcfg] = sys_getpmpcfg,
```

Enter를 어디서 세는가

pa2:kernel/console.c — consoleintr() 전체 관련 경로

```
147  uint nenter_count;
148
149  void
150  consoleintr(int c)
151  {
152    acquire(&cons.lock);
153
154    switch (c) {
155    case C('P'): // Print process list.
156      procdump();
157      break;
158    case C('U'): // Kill line.
159      while (cons.e != cons.w &&
160             cons.buf[(cons.e - 1) % INPUT_BUF_SIZE] != '\n') {
161        cons.e--;
162        consputc(BACKSPACE);
163      }
164      break;
165    case C('H'): // Backspace
166    case '\x7f': // Delete key
167      if (cons.e != cons.w) {
168        cons.e--;
169        consputc(BACKSPACE);
170      }
171      break;
172    default:
173      if (c != 0 && cons.e - cons.r < INPUT_BUF_SIZE) {
174        c = (c == '\r') ? '\n' : c;
175
176        if (c == '\n') nenter_count++;
177
178        // echo back to the user.
179        consputc(c);
180
181        // store for consumption by consoleread().
182        cons.buf[cons.e++ % INPUT_BUF_SIZE] = c;
183
184        if (c == '\n' || c == C('D') || cons.e - cons.r == INPUT_BUF_SIZE) {
185          // wake up consoleread() if a whole line (or end-of-file)
186          // has arrived.
187          cons.w = cons.e;
188          wakeup(&cons.r);
189        }
190      }
191      break;
192    }
193
194    release(&cons.lock);
```

CR(₩r)은 LF(₩n)으로 바뀐 뒤 검사된다. C('H')/DEL로 지운 입력은 카운트되지 않는다. 입력 버퍼가 가득 차 default 분기의 수용 조건에 실패하면 그 Enter도 세지 않는다. 입력 처리는 cons.lock 안에 있으므로 증가 동작은 직렬화된다.

pa2:kernel/sysproc.c — nenter()의 반환

```
115  extern uint nenter_count;
116
117  uint64 sys_nenter(void) {
118
119    return nenter_count;
120  }
```

주의: 읽기 쪽 sys_nenter()는 cons.lock을 잡지 않는다. 제출 구현을 설명할 때 읽기까지 동일한 잠금으로 보호했다고 쓰면 틀리다.

Q. 키보드 Enter가 모두 nenter에 반영되는가?

A. 아니다. 0이 아닌 문자이고 입력 버퍼에 공간이 있어 consoleintr()의 default 경로에 받아들여진 개행만 센다.

S-mode에서 M-mode로

시스템 콜 검증과 M-mode 진입 준비

pa2:kernel/sysproc.c — 인자 검증·M-mode 래퍼 호출

```
122  extern uint64 mcall_getpmpaddr(int n);
123  extern uint64 mcall_getpmpcfg(int n);
124
125  uint64 sys_getpmpaddr(void) {
126
127    int n;
128    argint(0, &n);
129
130    if (n < 0 || n > 3) return -1;
131
132    return mcall_getpmpaddr(n);
133  }
134
135  uint64 sys_getpmpcfg(void) {
136
137    int n;
138
139    argint(0, &n);
140
141    if (n < 0 || n > 3) return -1;
142
143    return (int) mcall_getpmpcfg(n);
144  }
```

argint(0,&n;)은 첫 번째 인자를 가져온다. 유효 범위는 0–3이며 밖이면 -1을 돌려준다. 반환형은 uint64지만 -1 비트 패턴은 모든 비트가 1인 값으로 전달된다. getpmpcfg에서는 int로 캐스팅한 뒤 uint64 반환형으로 변환된다.

pa2:kernel/start.c — mtvec와 medeleg

```
33   w_mtvec((uint64) machinevec); // 미리 시작할 때 세팅을 해두어야 scause=9 ecall이 일어났을 때 mtvec -> pc로 옮겨간다.
34
35   // delegate all interrupts and exceptions to supervisor mode.
36   w_medeleg(0xfdff); // 9번째 bit이 0으로 되어 있어야 M 모드 진입 가능.
37   w_mideleg(0xffff);
```

medeleg의 bit 9는 'S-mode에서 발생한 ecall'을 뜻한다. 0xfdff는 0xffff에서 bit 9(0x0200)를 끈 값이므로 이 예외가 M-mode로 간다. mtvec는 진입 주소 machinevec로 설정한다. 일반 사용자 ecall의 cause는 8이며 기존 U→S 시스템 콜 경로는 유지된다.

실제 M-mode trap 코드: 진입·주소 읽기

pa2:kernel/kernelvec.S — machinevec 및 pmpaddr 분기

```
.globl machinevec
machinevec:

        # mepc 4 증가
        csrr t0, mepc
        addi t0, t0, 4
        csrw mepc, t0

        li t0, 24
        beq a7, t0, handle_getpmpaddr
        li t0, 25
        beq a7, t0, handle_getpmpcfg
        li a0, -1
        mret

handle_getpmpaddr:

        li t0, 0
        beq a0, t0, read_pmpaddr0
        li t0, 1
        beq a0, t0, read_pmpaddr1
        li t0, 2
        beq a0, t0, read_pmpaddr2
        li t0, 3
        beq a0, t0, read_pmpaddr3
        li a0, -1
        mret

read_pmpaddr0:
        csrr a0, pmpaddr0
        j process_pmpaddr

read_pmpaddr1:
        csrr a0, pmpaddr1
        j process_pmpaddr

read_pmpaddr2:
        csrr a0, pmpaddr2
        j process_pmpaddr

read_pmpaddr3:
        csrr a0, pmpaddr3
        j process_pmpaddr
```

pa2:kernel/kernelvec.S — PMP 주소 변환

```
process_pmpaddr:
        slli a0, a0, 2
        li  t0, 0x00FFFFFFFFFFFFFF
        and a0, a0, t0
        mret
```

mepc를 4 올려 ecall 명령 다음으로 돌아간다. a7의 24/25로 작업을 분기하고 a0의 인덱스로 CSR을 고른다. pmpaddr는 CSR 값을 2비트 왼쪽으로 이동한 뒤 56비트 마스크를 씌운다. 이 구현은 'pmpaddr CSR 원값' 그대로를 반환하지 않는다는 점을 기억한다.

PMP 설정 바이트와 경계 조건

pmpcfg0에서 인덱스별 바이트 추출

pa2:kernel/kernelvec.S — 설정 레지스터 뷰기·마스크

```
116  handle_getpmpcfg:
117          li t0, 0
118          beq a0, t0, read_pmpcfg0
119          li t0, 1
120          beq a0, t0, read_pmpcfg1
121          li t0, 2
122          beq a0, t0, read_pmpcfg2
123          li t0, 3
124          beq a0, t0, read_pmpcfg3
125          li a0, -1
126          mret
127
128  read_pmpcfg0:
129          csrr a0, pmpcfg0
130          li  t0, 0x00000000000000FF
131          and a0, a0, t0
132          mret
133
134  read_pmpcfg1:
135          csrr a0, pmpcfg0
136          li  t0, 0x000000000000FF00
137          and a0, a0, t0
138          srli a0, a0, 8
139          mret
140
141  read_pmpcfg2:
142          csrr a0, pmpcfg0
143          li  t0, 0x0000000000FF0000
144          and a0, a0, t0
145          srli a0, a0, 16
146          mret
147
148  read_pmpcfg3:
149          csrr a0, pmpcfg0
150          li  t0, 0x00000000FF000000
151          and a0, a0, t0
152          srli a0, a0, 24
153          mret
```

pa2:kernel/kernelvec.S — S-mode 래퍼 두 개

```
156  .globl mcall_getpmpaddr
157  mcall_getpmpaddr:
158
159          li a7, 24
160          ecall
161          ret
162
163  .globl mcall_getpmpcfg
164  mcall_getpmpcfg:
165
166          li a7, 25
167          ecall
168          ret
```

pmpcfg0의 하위 32비트 안에 설정 바이트 0–3이 차례로 들어간다. 인덱스 0은 bits 7:0, 1은 15:8, 2는 23:16, 3은 31:24를 마스크하고 오른쪽으로 이동한다. 예를 들어 pmpcfg0 = 0xA1B2C3D4이면 getpmpcfg(2)는 0xB2다.

시험에서 설명할 근거

- **왜 M-mode?** PMP CSR은 머신 권한의 보호 설정이다. S-mode에서 직접 csrr하면 접근 권한 문제가 생기므로 ecall로 M-mode trap handler를 실행한다.
- **왜 mepc+4?** RISC-V의 ecall은 trap을 일으킨 명령을 가리키는 mepc를 남긴다. 건너뛰지 않으면 mret 뒤 같은 ecall을 반복한다.
- **인자·반환 레지스터:** a0에 n이 들어가고 읽은 값도 a0로 돌아온다. a7은 래퍼에서 서비스 번호 24/25로 설정한다.

- **구현 경계:** machinevec는 별도 레지스터 저장 스택을 구성하지 않는다. 제출 코드의 짧은 래퍼·핸들러 경로를 그대로 설명하고, 일반적인 모든 M-mode trap을 처리한다고 확대하지 않는다.

Q. getpmpaddr(4)를 호출하면 어떤 경로를 지나는가?

A. U→S syscall까지는 오지만 sys_getpmpaddr()의 0–3 검사에서 -1을 반환한다. S→M ecall은 실행하지 않는다.

Q. medeleg을 다시 0xffff로 두면?

A. S-mode ecall의 예외 위임 bit 9가 켜져 의도한 machinevec 진입을 보장하지 못한다. PMP 조회 경로가 깨진다.

PA2 손계산·코드 추적 연습

Q. pmpcfg0 = 0xA1B2C3D4일 때 n=0,1,2,3의 반환값은?

A. 각각 0xD4, 0xC3, 0xB2, 0xA1. getpmpcfg()는 pmpcfg0에서 n번째 바이트를 마스크·시프트한다.

Q. pmpaddr2 CSR 원값이 0x1234이면 getpmpaddr(2)는?

A. 0x1234 << 2 = 0x48D0. 그 뒤 0x00FFFFFFFFFFFFFF 마스크를 적용하며 이 예에서는 값이 그대로다.

Q. machinevec에 a7=26으로 들어가면?

A. 24/25 어느 분기도 맞지 않아 a0=-1을 놓고 mret한다. 이 코드는 다른 M-mode trap을 일반적으로 처리하는 핸들러가 아니다.

- **암기보다 경로:** 사용자 stub의 a7 → S-mode syscall 표 → sysproc의 범위 검사 → 래퍼의 a7 재설정 → M-mode CSR 읽기 → a0 반환. 두 a7 값의 수치가 같아도 서로 다른 trap 경로에서 쓰인다.

2.3 PA3 — EDF 실시간 스케줄러

요구사항과 상태

실시간(RT) 작업에는 runtime과 period를 지정하고, deadline이 가장 가까운 RUNNABLE RT 작업을 우선한다. deadline 동률이면 직전에 실행하던 작업, 그다음 PID가 작은 작업을 택한다. RT가 없을 때 일반 프로세스는 순환하며 실행한다. RT 작업의 sched_yield()는 이번 job이 끝났음을 표시하고 다음 period까지 쉰다.

pa3:kernel/proc.h — 프로세스의 RT 메타데이터

```
 92    int runtime;
 93    int period;
 94    uint deadline;
```

pa3:kernel/proc.c — allocproc 초기값

```
128    p->runtime = 0;
129    p->period = 0;
130    p->deadline = 0;
```

pa3:kernel/syscall.h — API 번호

```
 25  #define SYS_sched_setattr 30
 26  #define SYS_sched_yield   31
```

pa3:kernel/sysproc.c — syscall 인자·범위 검사

```
115  uint64
116  sys_sched_setattr(void)
117  {
118
119    // FILL HERE
120    int pid;
121    int runtime;
122    int period;
123
124    argint(0, &pid);
125    argint(1, &runtime);
126    argint(2, &period);
127
128    if (runtime >= period) return -1;
129    if (runtime <= 0) return -1;
130    if (period <= 0) return -1;
131
132    return ksched_setattr(pid, runtime, period);
133  }
134
135  uint64
136  sys_sched_yield(void)
137  {
138
139    ksched_yield();
140    return 0;
141  }
```

runtime ≥ period, runtime ≤ 0, period ≤ 0이면 -1이다. pid=0은 호출 프로세스를 뜻한다. 유효한 값은 ksched_setattr()로 넘어간다. syscall 번호 30/31의 표 등록과 user stub을 통해 사용자 코드가 진입한다.

pa3:kernel/proc.c — PID 검색과 절대 deadline 설정

```
767  ksched_setattr(int pid, int runtime, int period) {
768
769    struct proc *p;
770    uint now;
771
772    if (pid == 0) pid = myproc()->pid;
773
774    acquire(&tickslock);
775    now = ticks;
776    release(&tickslock);
777
778    for (p = proc; p < &proc[NPROC]; p++) {
779
780      acquire(&p->lock);
781
782      if (p->pid == pid && p->state != UNUSED) {
783        p->runtime = runtime;
784        p->period = period;
785        p->deadline = now + period;
786        release(&p->lock);
787        return 0;
788      }
789
790      release(&p->lock);
791    }
792
793    return -1;
794  }
```

deadline=현재 ticks+period이다. 이 값은 각 job의 상대 제한 시간이 아니라 저장된 절대 시각이다. 대상 PID가 없으면 -1이다. 매번 프로세스 락을 잡고 해당 프로세스를 찾는다.

완료 보고와 다음 주기

pa3:kernel/proc.c — RT 완료 / 일반 양보

```
796  void ksched_yield(void) {
797
798    struct proc *p = myproc();
799
800    acquire(&p->lock);
801
802    if (p->runtime > 0 && p->period > 0) {
803      p->state = SLEEPING;
804    } else {
805      p->state = RUNNABLE;
806    }
807
808    sched();
809    release(&p->lock);
810
811  }
```

RT이면 SLEEPING, 일반 프로세스이면 RUNNABLE로 바꾸고 sched()로 CPU를 내놓는다. 이 SLEEPING은 일반 wait channel에 등록한 sleep()과 다르다. 다음 scheduler 순회가 ticks와 deadline을 비교해 RT 작업을 직접 깨운다.

스케줄러 전반부: 잠든 RT 작업 깨우기와 EDF 선택

pa3:kernel/proc.c — scheduler 시작·EDF 후보 검색

```
432  void
433  scheduler(void)
434  {
435    struct proc *p;
436    struct cpu *c = mycpu();
437
438    //deadline이 같을 때 이전에 실행하던 process우선
439    struct proc *previous = 0;
440    int next = 0;
441
442    c->proc = 0;
443    for (;;) {
444      // The most recent process to run may have had interrupts
445      // turned off; enable them to avoid a deadlock if all
446      // processes are waiting. Then turn them back off
447      // to avoid a possible race between an interrupt
448      // and wfi.
449      intr_on();
450      intr_off();
451
452
453      // RT 프로세스 다시 깨우기 & 가장 deadline이 빠른 proc 찾기
454      uint now;
455
456      uint best_deadline = 0;
457      struct proc *best = 0;
458      int best_pid = 0;
459
460      acquire(&tickslock);
461      now = ticks;
462      release(&tickslock);
463
464      for (p = proc; p < &proc[NPROC]; p++) {
465
466        acquire(&p->lock);
467
468        if (p->period > 0 && p->runtime > 0) {
469          if (p->state == SLEEPING && now >= p->deadline) {
470            p->deadline += p->period;
471            p->state = RUNNABLE;
472          }
473          if (p->state == RUNNABLE) {
474            if (best== 0 || p->deadline < best_deadline || (p->deadline == best_deadline && best !
     = previous && (p == previous || p->pid < best_pid))) {
475              best_deadline = p->deadline;
476              best = p;
477              best_pid = p->pid;
478            }
479          }
480        }
481
482        release(&p->lock);
483      }
484      //
485
```

순회마다 ticks를 읽고 각 프로세스의 lock을 잡는다. period/runtime이 양수인 RT가 SLEEPING이고 now ≥ deadline이면 deadline에 period를 한 번 더하고 RUNNABLE로 전환한다. 이후 RUNNABLE RT 중 가장 이른 deadline을 고른다.

동률 조건식은 'previous가 후보가 아니고, 새 p가 previous이거나 PID가 더 작을 때' 후보를 교체한다. 따라서 현재 후보가 previous이면 다른 PID가 작아도 previous가 유지된다.

Q. deadline이 같은 A(PID 9)·B(PID 3)가 있고 직전 실행 프로세스가 A면?

A. A를 택한다. previous가 둘 중 없다면 PID가 작은 B를 택한다.

RT 우선과 일반 RR

pa3:kernel/proc.c — RT 문맥 교환·일반 RR·idle

```
486     if (best != 0) {
487       acquire(&best->lock);
488       best->state = RUNNING;
489       c->proc = best;
490       swtch(&c->context, &best->context);
491
492       mycpu()->intena = 0;
493
494       previous = best;
495       c->proc = 0;
496       release(&best->lock);
497       continue;
498     }
499
500     int found = 0;
501     for (int i = 0; i < NPROC; i++) {
502       int index = (next + i) % NPROC;
503       p = &proc[index];
504
505       acquire(&p->lock);
506       if (p->state == RUNNABLE) {
507         // Switch to chosen process.  It is the process's job
508         // to release its lock and then reacquire it
509         // before jumping back to us.
510         p->state = RUNNING;
511         c->proc = p;
512         swtch(&c->context, &p->context);
513
514         // Don't re-enable interrupts on release.
515         mycpu()->intena = 0;
516
517         // Process is done running for now.
518         // It should have changed its p->state before coming back.
519         next = (index + 1) % NPROC;
520         previous= p;
521         c->proc = 0;
522         found = 1;
523         release(&p->lock);
524         break;
525       }
526       release(&p->lock);
527     }
528     if (found == 0) {
529       // nothing to run; stop running on this core until an interrupt.
530       asm volatile("wfi");
531     }
532   }
533 }
```

RT 후보가 있으면 해당 p를 RUNNING으로 바꾸고 c->proc를 설정한 뒤 swtch()한다. 돌아오면 previous를 기억한다. RT가 없을 때는 next 인덱스부터 프로세스 테이블을 원형으로 찾아 RUNNABLE을 실행한다. 선택한 일반 작업의 다음 인덱스를 next로 저장한다. 아무 프로세스도 없으면 wfi로 인터럽트를 기다린다.

시간 흐름 예제

A=(runtime 1, period 3), B=(1,4)를 같은 tick 0에 등록했다면 첫 절대 deadline은 A=3, B=4다. A가 먼저 1단위 작업을 마치고 sched_yield()하면 SLEEPING이 된다. 다음에 B가 실행된다. tick이 A의 deadline 3에 이르면 A는 deadline 6으로 갱신되며 RUNNABLE이 된다. 이 설명은 개념적 추적이며 실제 로그의 wall-clock 간격은 계산량·timer interrupt 시점에 영향을 받는다.

Q. RT A가 sched_yield()를 호출하지 않으면 runtime 1 뒤에 자동으로 쉬는가?

A. 아니다. 제출 코드는 runtime 잔여량을 감산하지 않는다. timer interrupt는 yield()로 재스케줄링하지만, job 완료 전환은 sched_yield() 호출에 의존한다.

Q. RT가 RUNNABLE인 동안 일반 프로세스가 선택되는가?

A. scheduler의 RT 후보 분기가 먼저이므로 이 구현에서 RT 후보가 있으면 일반 RR 분기에 도달하지 않는다.

테스트 코드와 시험 해석

과제 테스트의 작업 생성

pa3:user/task1.c — 두 주기 작업의 등록

```
57  int
58  main(int argc, char *argv[])
59  {
60    uint64 x = 0;
61    int pid1, pid2;
62
63    pause(1);
64    if ((pid1 = fork()))
65    {
66      if ((pid2 = fork()))
67      {
68        sched_setattr(pid1, 1, 3);
69        sched_setattr(pid2, 1, 4);
```

pa3:user/task2.c — 세 주기 작업의 등록

```
70          sched_setattr(pid1, 1, 8);
71          sched_setattr(pid2, 2, 5);
72          sched_setattr(pid3, 4, 10);
```

pa3:user/task1.c — 각 job이 CPU 사용 후 yield

```
43  do_fork(int n)
44  {
45    int x = 0;
46
47    for (int i = 0; i < 10; i++)
48    {
49      printf("%lu %d starts\n", time()/100000, getpid());
50      x += run_task(n);
51      printf("%lu %d ends\n", time()/100000, getpid());
52      sched_yield();
53    }
54    return x;
55  }
```

task1의 사용률 합은 1/3+1/4=7/12 0.583이다. task2는 1/8+2/5+4/10=0.925다. 둘 다 1 이하이므로 단일 CPU의 이상적인 주기 작업 EDF 모델에서 실행 가능성 검토에 쓰는 예다. 그러나 제출 코드는 사용률 입장 심사나 runtime 강제 제한을 구현하지 않으므로 이 계산만으로 실제 실행의 마감 보장을 증명할 수는 없다.

fork와 상태 초기화

allocproc()에서 RT 필드를 0으로 초기화하므로 fork로 생긴 자식은 부모의 RT 속성을 그대로 상속하지 않는다. 부모가 sched_setattr(child_pid, ...)을 호출해 각 자식을 RT로 지정한다. 이 순서가 task1/task2 코드에 드러난다.

제출 코드에서 정확히 말할 한계

- runtime 값은 속성 검사와 RT 판별에 쓰지만 실행 시간 차감·초과 정지에는 사용되지 않는다.
- 깨우기 조건 now ≥ deadline에서 deadline에 period를 한 번만 더한다. 여러 주기를 놓친 경우 즉시 과거 deadline을 모두 건너뛰는 루프는 없다.
- EDF 후보 검색 후 lock을 풀고 다시 best lock을 잡는다. 선택 뒤 상태 재검사가 없어 다중 CPU 동시 스케줄링에는 경합 위험이 있다. 이 과제의 기본 단일 CPU 맥락에서 설명한다.
- RT의 SLEEPING 상태를 scheduler가 deadline 기준으로 직접 깨우므로, 일반 sleep/wakeup과 의미가 다르다. RT가 다른 이유로 잠드는 확장 상황은 별도 구분이 필요하다.

2부 마무리 — 빈 종이 재현 문제

Q. PA1: 학기 문구가 출력되는 함수·모드·시점은?

A. kernel/main.c의 main(), 커널 S-mode, console/printk 초기화 직후이며 userinit() 이전이다.

Q. PA2: nenter가 증가하는 정확한 조건은?

A. consoleintr()의 default 분기에서 입력이 버퍼에 수용되고 CR이 LF로 정규화된 뒤 c=='₩n'일 때다.

Q. PA2: PMP 레지스터를 읽는 두 번째 ecall에서 a0/a7/mepc의 역할은?

A. a0는 인덱스이자 반환값, a7은 24/25 서비스 번호, mepc는 mret 뒤 ecall 다음 명령으로 가도록 +4 한다.

Q. PA3: 같은 deadline을 가진 세 작업 중 직전 작업이 있다면?

A. 직전 작업을 우선한다. 직전 작업이 후보에 없을 때 PID가 작은 작업을 고른다.

Q. PA3: 일반 yield와 RT sched_yield의 상태 차이는?

A. 일반은 RUNNABLE, RT는 SLEEPING이다. 후자는 deadline이 되면 scheduler가 RUNNABLE로 만든다.

Q. PA3: runtime=2, period=5를 등록하면 실행 시간이 2 tick으로 제한되는가?

A. 아니다. 제출 구현에는 runtime을 측정·감산하는 로직이 없다. 테스트 작업은 스스로 sched_yield()를 호출한다.

원자료: 강의·손필기 = 이 PDF의 1부 / 과제 명세 = github.com/snu-csl/os-pa1, os-pa2, os-pa3 / 제출 코드 = Git pa1·pa2·pa3 브랜치. 코드 줄번호는 해당 브랜치의 파일 기준이다.